Composite, Covering & Partial Indexes

column order, the leftmost prefix and indexes that fit the query

Watch the 3-min video

In one line: a composite index is sorted column by column, so only a leftmost prefix can use it (equality columns first, range or sort last); a covering index answers from the index alone; a partial index holds only the rows you query.

1 One table, different query shapes

A job queue table has **50 million rows**, but only a few hundred are *pending* at any moment, and workers poll for them every second. Meanwhile a reports page filters orders by status and date. A plain single-column index is the wrong tool for both. Three index shapes cover most real needs: **composite**, **covering** and **partial**.

2 Composite indexes and the leftmost prefix rule

A **composite index** covers several columns. It is sorted by the first column, then by the second within each first value, and so on, like a phone book sorted by last name, then first name.

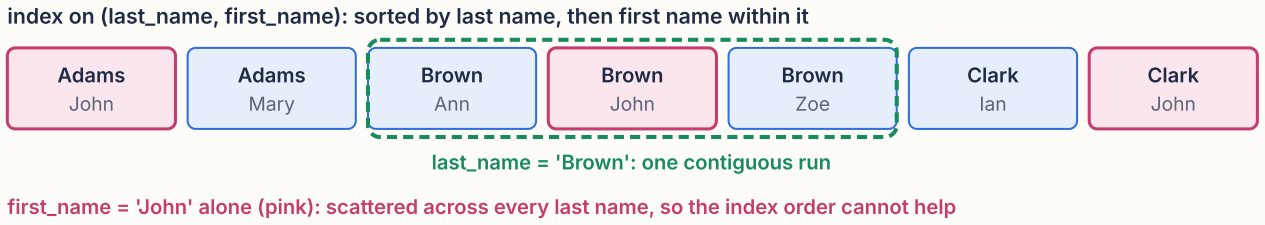


Figure 1. A phone book finds “Brown” instantly, and “Brown, John” even faster, but it cannot help you find everyone named John.

That is the **leftmost prefix rule**. An index on `(a, b, c)` can serve:

Query filters on	Uses the index well?
`a`	Yes
`a` and `b`	Yes
`a`, `b` and `c`	Yes
`a` and `c`	Partly: seeks on `a`, then checks `c` entry by entry
`b` alone, or `c` alone	No: not a leftmost prefix

3 Column order: equality first, range or sort last

For `status = 'shipped' AND created_at > last week`, index **(status, created_at)**. The database jumps to *shipped*, and inside it the dates are sorted, so it reads **one contiguous range**. Reverse the order, and the shipped rows are scattered across the whole date range, mixed with every other status.



Figure 2. Equality column first turns the query into one seek plus one short range. Range column first means reading and discarding the other statuses.

WORKED EXAMPLE

Orders: **20 million** rows, 100,000 from the last week, 80% of all orders shipped. The reports page asks for last week's *shipped* orders (about 80,000 rows).

- **(status, created_at)**: one seek, then exactly the ~80,000 matching entries.
- **(created_at, status)**: one seek, then all ~100,000 recent entries, discarding 20,000. Not terrible here.
- Now ask for last week's *returned* orders (1%, about 1,000 rows). Status-first reads 1,000 entries; date-first still reads 100,000, so **100× more work**.

The more selective the equality filter, the more the order matters.

4 Covering indexes: answer from the index alone

A **covering index** contains every column the query needs, so the database answers from the index alone: an **index-only scan**, with no trips to the table. In Postgres (11+) and SQL Server, `INCLUDE` adds payload columns to the leaf pages without making them part of the sort key. (MySQL InnoDB has no INCLUDE: append the column to the key instead.)

```
CREATE INDEX idx_orders_cust ON orders (customer_id, created_at) INCLUDE (total);

-- served by an Index Only Scan:
SELECT created_at, total FROM orders
WHERE customer_id = 17 ORDER BY created_at DESC LIMIT 50;
```

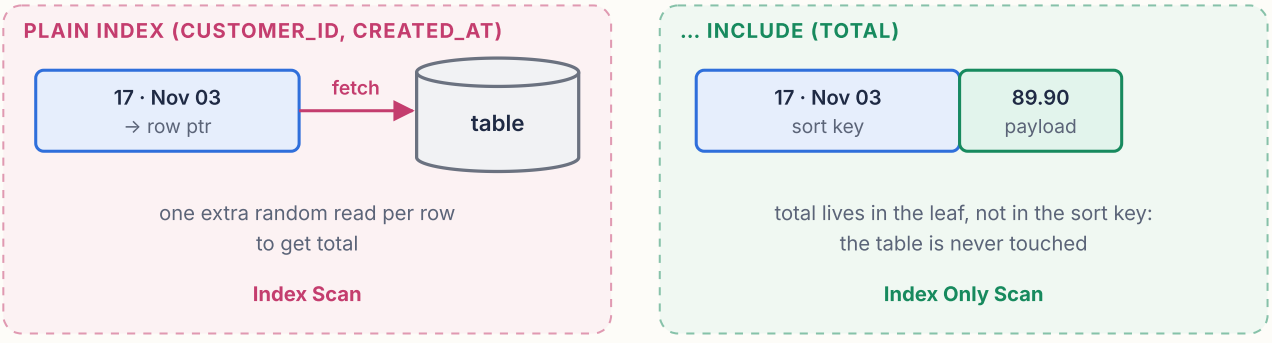


Figure 3. Listing a customer's orders with totals never touches the table. Included columns are not sorted, so they cannot be used to filter by the index order, only to return values. (In Postgres the visibility map must also show the pages as all-visible, which VACUUM maintains.)

5 Partial and expression indexes

A **partial index** only covers rows that match a condition. For the job queue:

```
CREATE INDEX idx_jobs_pending ON jobs (run_at) WHERE status = 'pending';

-- the query must repeat the same condition, so the planner can match it:
SELECT id FROM jobs WHERE status = 'pending' AND run_at <= now()
ORDER BY run_at LIMIT 10;
```

Instead of 50 million entries, it holds a few hundred. It is tiny, stays in memory, and is cheap to maintain, because finished jobs drop out of it.

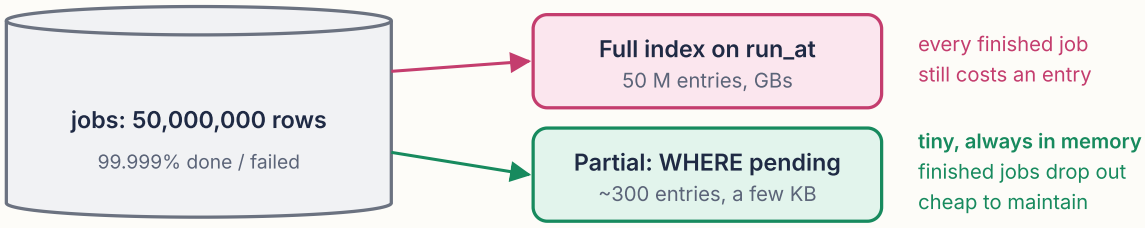


Figure 4. The partial index only stores what the workers actually poll for.

An **expression index** indexes a computed value. If you query `WHERE lower(email) = 'a@b.com'`, a plain index on `email` cannot help. Create it on the expression instead: `CREATE INDEX ON users (lower(email));`. The same works for truncated dates (`date_trunc('day', created_at)`) or a field extracted from JSON (`(data->>'country')`).

6 In practice

1. Start from your **slowest and most frequent** queries, not from the table schema.
2. Read the plan with `EXPLAIN ANALYZE`. Look for sequential scans on large tables and sorts you could avoid.
3. Design **one index that serves several queries** using the leftmost prefix rule: `(customer_id, created_at)` also serves `WHERE customer_id = ?`, so a separate index on `customer_id` is redundant.
4. Keep an eye on the **write overhead**: every index is updated on every insert.

COMMON MISTAKES

- Putting the range column first: `(created_at, status)` for an equality-plus-range query.
- Creating one index per query, including redundant prefixes like `(a)` next to `(a, b)`.
- A partial index whose condition the query does not repeat: the planner cannot prove it applies, so it is ignored.
- Querying `lower(email)` against a plain index on `email`.

7 Key terms

Composite index: index on several columns, sorted left to right

Covering index: holds every column the query needs

Index-only scan: answered without reading the table

Expression index: indexes a computed value, e.g. lower(email)

Leftmost prefix: only leading columns can drive the seek

INCLUDE: payload columns stored in leaves, not sorted

Partial index: covers only rows matching a WHERE

Selectivity: fraction of rows a filter keeps

8 Interview questions

Q1. Why does column order matter in a composite index? BASIC

The index is sorted by the first column, then the next, so only queries using a leftmost prefix can use it efficiently. Put equality columns first, and range or sort columns last.

Q2. What is a covering index? BASIC

An index that contains every column a query needs, often through INCLUDE, so the query is answered with an index-only scan without reading the table.

Q3. An index exists on (a, b, c). Which of these use it: WHERE b = 1; WHERE a = 1 AND c = 2; WHERE a = 1 ORDER BY b? BASIC

WHERE b = 1: no, b is not a leftmost prefix. a and c: partly, it seeks on a and then checks c entry by entry. a = 1 ORDER BY b: yes, fully, and the sort is free because b is already ordered within a.

Q4. Workers poll a 50M-row jobs table every second for pending jobs and the query is slow. What do you add? SCENARIO

A partial index: CREATE INDEX ON jobs (run_at) WHERE status = 'pending'. It holds a few hundred entries instead of 50 million, stays in memory, and the polling query (with the same WHERE) reads a handful of entries.

Q5. Login does WHERE lower(email) = ? and scans 10M users even though email is indexed. Why, and the fix? SCENARIO

The index stores email, not lower(email), so it cannot match the expression. Create an expression index on (lower(email)), or store emails normalized to lower case and query the plain column.

Q6. You have separate indexes on customer_id and on (customer_id, created_at). Keep both? SCENARIO

Drop the single-column one. The composite's leftmost prefix already serves WHERE customer_id = ?, and every extra index costs a write on every insert.

Q7. Why not put every column of a hot query in the key instead of using INCLUDE? TRICKY

Key columns make the index wider at every level and must be kept sorted. INCLUDE columns only sit in the leaves, so internal pages stay small and fan-out stays high. They cannot help filtering or sorting, but they still enable an index-only scan.

Q8. EXPLAIN shows Index Only Scan but with many "Heap Fetches". Why? TRICKY

In Postgres the index has no visibility info, so for pages not marked all-visible in the visibility map it still checks the table. Run or tune VACUUM on write-heavy tables so more pages are all-visible.

Next: SD-E-006 · LSM Trees & SSTables backendroadmap.com